

RTL LAB WEEK#2

SUBMITTED TO:

SIR KHALIL REHMAN

SUBMITTED BY:

LAREB FARHAN

Tasks Performed :

- T-flip flop
- D-flip flop
- JK Flip flop/SR flipflop
- 4 bit synchronous up counter
- 4 bit synchronous down counter
- 4 bit asynchronous up counter
- 4 bit asynchronous down counter
- bcd counter
- mod n counter
- frequency divider (100MHZ - 25 MHZ)
- alu using pipeline
- multiplier using pipeline
- Johnson counter
- ring counter
- SISO
- SIPO
- PISO
- PIPO

Tas#1: D-Flip flop**Verilog Code :**

```
module d_ff(  
    input clk,  
    input rst,  
    input d,  
    output reg q  
);  
  
always @(posedge clk or posedge rst) begin  
    if (rst)  
        q <= 1'b0;  
    else  
        q <= d;  
    end  
  
endmodule
```

Test Bench:

```
`timescale 1ns/1ps
```

```

module tb_d_ff;
reg clk;
reg rst;
reg d;
wire q;

d_ff dut (
    .clk(clk),
    .rst(rst),
    .d(d),
    .q(q)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_d_ff);
end
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    d = 0;

    #10 rst = 0;

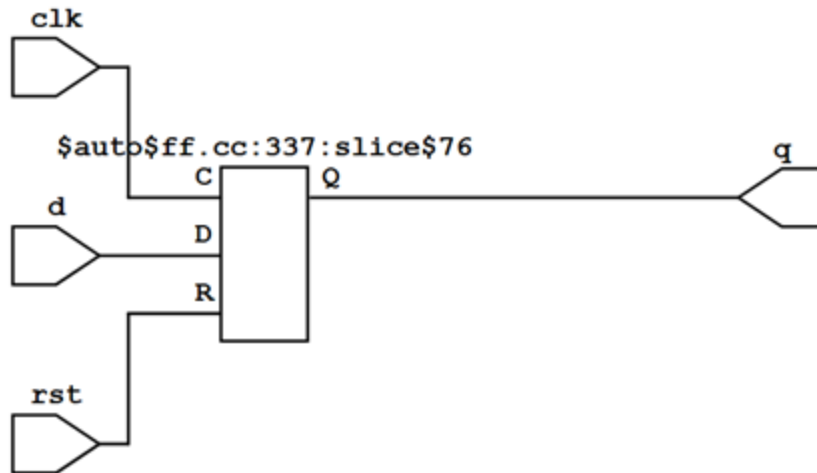
    d = 1; #10;
    d = 0; #10;
    d = 1; #10;

    #10 $finish;
end
endmodule

```

Wave Form :





Tas#2: T-Flip flop

Verilog Code :

```
module t_ff(
    input clk,
    input rst,
    input t,
    output reg q
);
```

```
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else if (t)
            q <= ~q;
        else
            q <= q;
    end
endmodule
```

Test Bench:

```
`timescale 1ns/1ps
```

```
module tb_t_ff;
```

```
    reg clk;
    reg rst;
    reg t;
    wire q;
```

```
    t_ff uut (
```

```

.clk(clk),
.rst(rst),
.t(t),
.q(q)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_t_ff);
end

```

```
always #5 clk = ~clk;
```

```

initial begin
    clk = 0;
    rst = 1;
    t = 0;

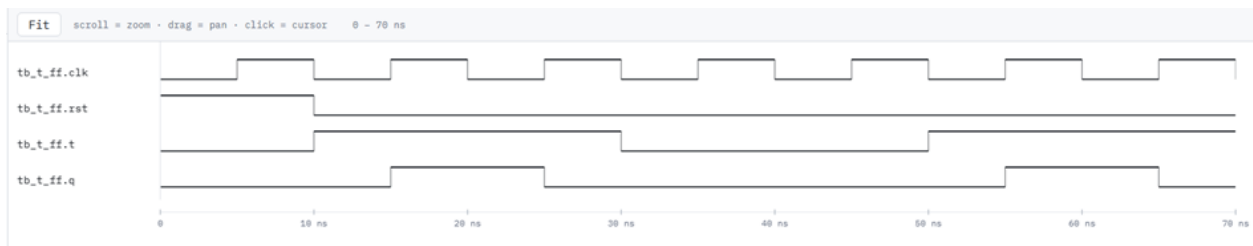
    #10 rst = 0;

    t = 1; #20; // toggle on two clock edges
    t = 0; #20; // hold value
    t = 1; #20; // toggle on two clock edges

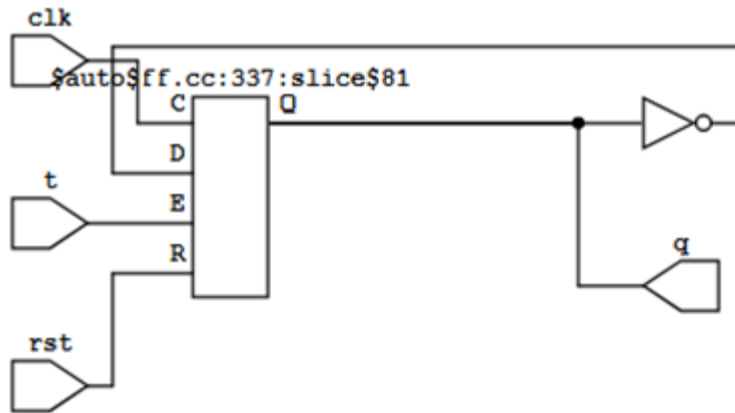
    $finish;
end

```

endmodule
Wave Form:



Synthesis:



Task#3: Jk-Flipflop:

Verilog Code:

```

module jk_ff(
    input clk,
    input rst,
    input j,
    input k,
    output reg q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 1'b0;
    else begin
        case ({j,k})
            2'b00: q <= q; // No change
            2'b01: q <= 1'b0; // Reset
            2'b10: q <= 1'b1; // Set
            2'b11: q <= ~q; // Toggle
        endcase
    end
end
endmodule

```

Test bench:

```

`timescale 1ns/1ps

module tb_jk_ff;

reg clk, rst, j, k;

```

```

wire q;

jk_ff uut (
    .clk(clk),
    .rst(rst),
    .j(j),
    .k(k),
    .q(q)
);

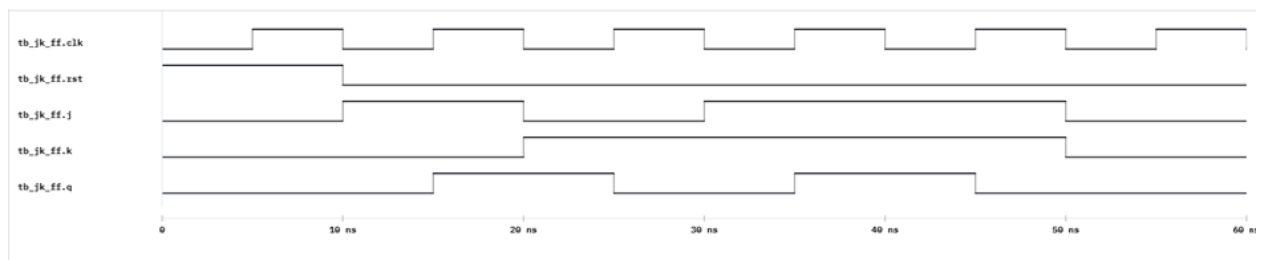
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_jk_ff);
end
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    j = 0;
    k = 0;

    #10 rst = 0;

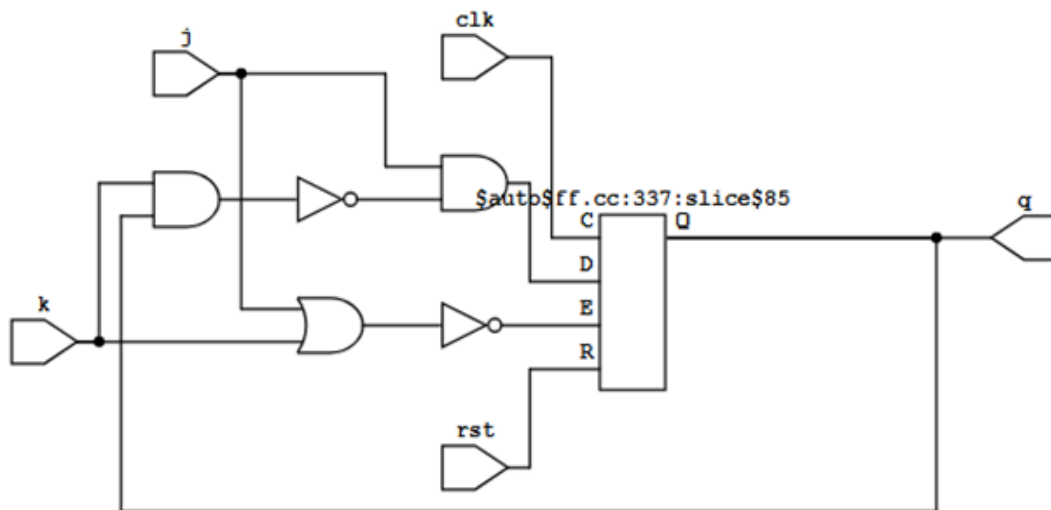
    j = 1; k = 0; #10; // Set
    j = 0; k = 1; #10; // Reset
    j = 1; k = 1; #20; // Toggle twice
    j = 0; k = 0; #10; // Hold

    $finish;
end
endmodule
WaveForm:

```



Synthesis:



Task#4:SR-Flipflop

Verilog Code:

```
module sr_ff(  
    input clk,  
    input rst,  
    input s,  
    input r,  
    output reg q  
);
```

```
always @(posedge clk or posedge rst) begin
```

```
    if (rst)
```

```
        q <= 1'b0;
```

```
    else begin
```

```
        case ({s,r})
```

```
            2'b00: q <= q;
```

```
            2'b01: q <= 1'b0;
```

```
            2'b10: q <= 1'b1;
```

```
            2'b11: q <= 1'bx; // invalid state
```

```
        endcase
```

```
    end
```

```
end
```

```
endmodule
```

Test Bench :

```
`timescale 1ns/1ps
```

```
module tb_sr_ff;
```



```

reg clk, rst, s, r;
wire q;

sr_ff uut (
    .clk(clk),
    .rst(rst),
    .s(s),
    .r(r),
    .q(q)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_sr_ff);
end

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    s = 0;
    r = 0;

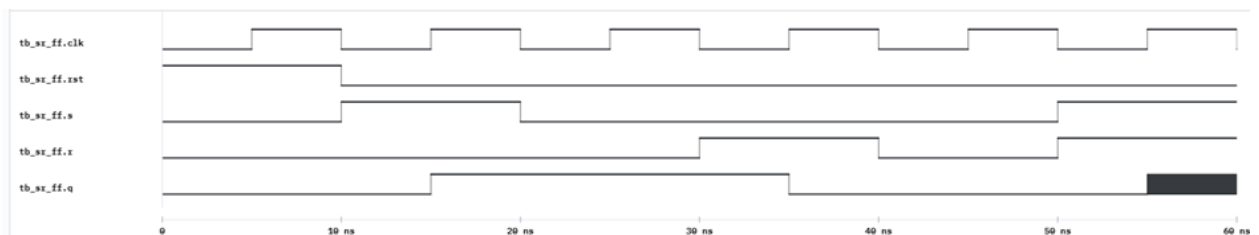
    #10 rst = 0;

    s = 1; r = 0; #10; // Set
    s = 0; r = 0; #10; // Hold
    s = 0; r = 1; #10; // Reset
    s = 0; r = 0; #10; // Hold
    s = 1; r = 1; #10; // Invalid

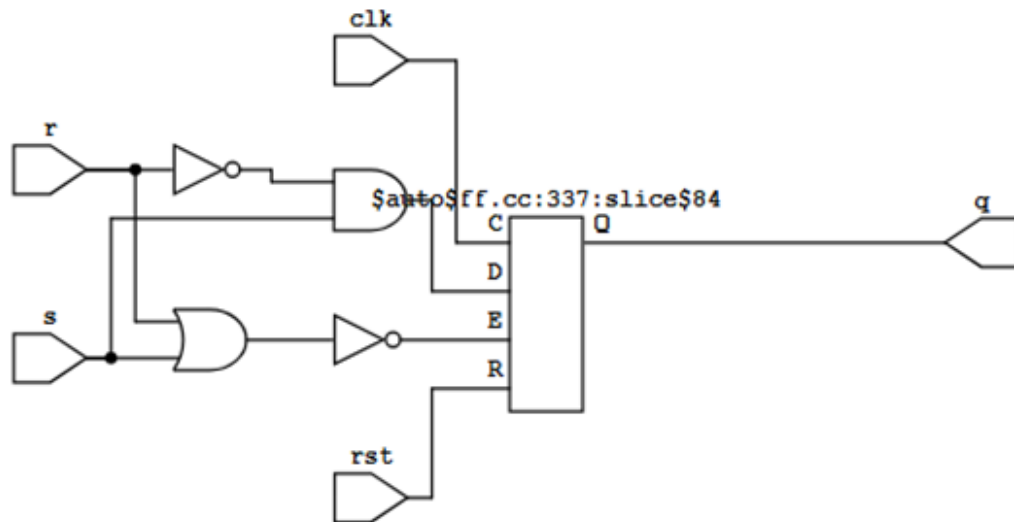
    $finish;
end

endmodule
Wave Form:

```



Synthesis:



Task#5: 4 bit synchronous up counter

Verilog code :

```
module up_counter_4bit(
    input clk,
    input rst,
    output reg [3:0] q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 4'b0000;
    else
        q <= q + 1'b1;
end
```

endmodule

Test bench:

```
`timescale 1ns/1ps
```

```
module tb_up_counter_4bit;
```

```
reg clk, rst;
```

```
wire [3:0] q;
```

```
up_counter_4bit uut (
```

```
    .clk(clk),
```

```
    .rst(rst),
```

```
    .q(q)
```

```
);
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0, tb_up_counter_4bit);
```

```
end
```

```
always #5 clk = ~clk;
```

```
initial begin
```

```
    clk = 0;
```

```
    rst = 1;
```

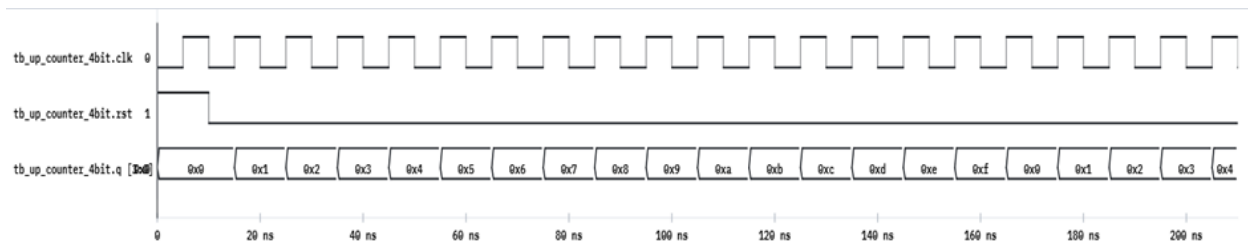
```
    #10 rst = 0;
```

```
    #200 $finish;
```

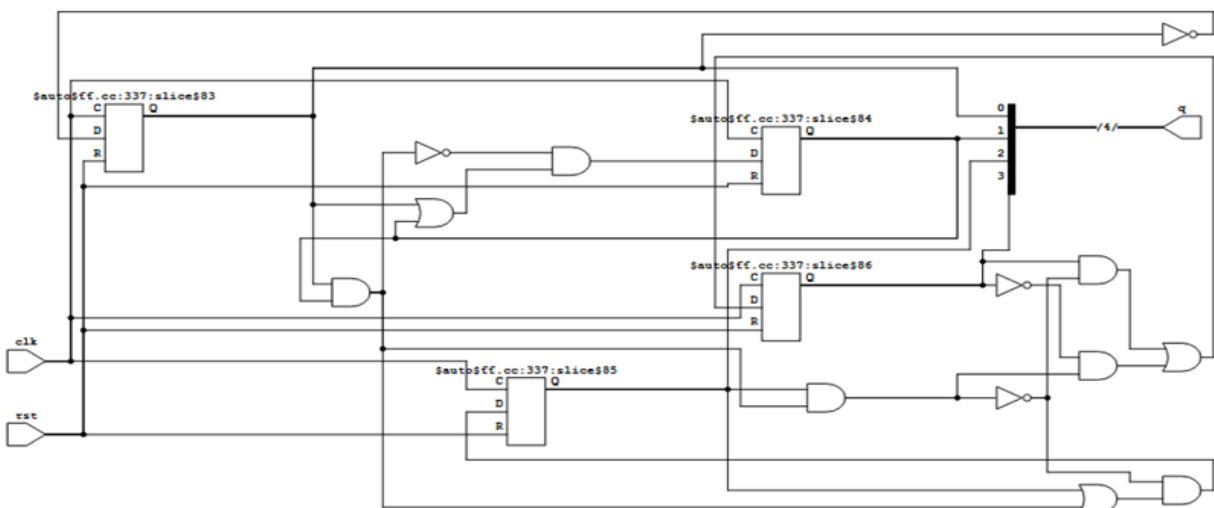
```
end
```

```
endmodule
```

Wave form:



Synthesis:



Task#5:4 bit synchronous down counter

Verilog Code:

```
module down_counter_4bit(
    input clk,
    input rst,
    output reg [3:0] q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= 4'b1111; // Start from 15
    else
        q <= q - 1'b1;
end
endmodule
```

Test bench :

```
`timescale 1ns/1ps
module tb_down_counter_4bit;

    reg clk, rst;
    wire [3:0] q;

    down_counter_4bit uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_down_counter_4bit);
    end

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;

        #10 rst = 0;

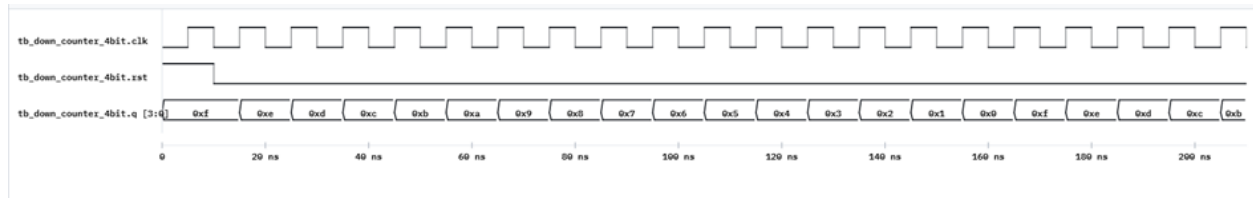
        #200 $finish;
    end
endmodule
```

```
end
endmodule
```

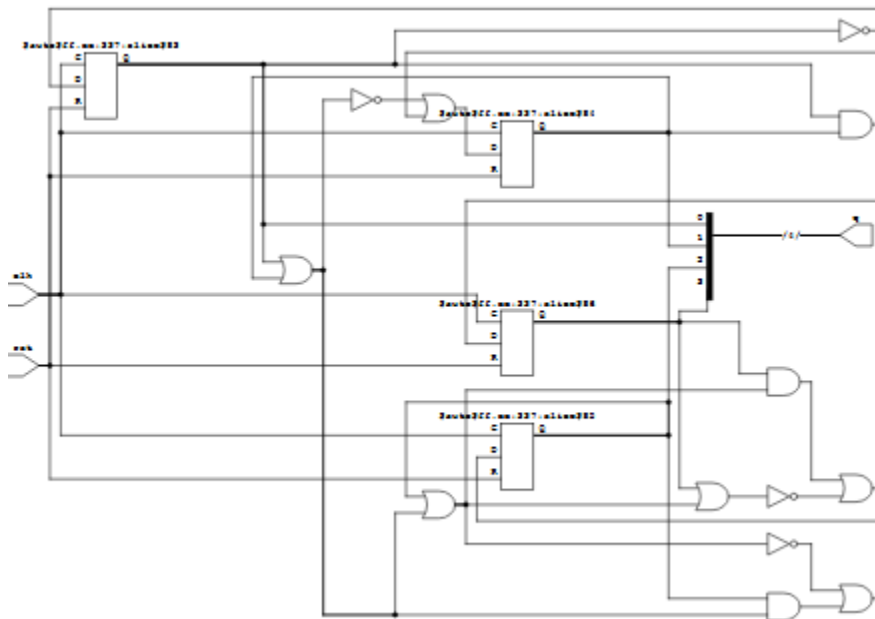
Wave

form

:



Synthesis:



Task#6:4 bit asynchronous up counter

Verilog Code:

```
module async_up_counter_4bit(
    input clk,
    input rst,
    output reg [3:0] q
);
```

```
always @(posedge clk or posedge rst)
```

```
begin
```

```
    if (rst)
```

```
        q[0] <= 0;
```

```
    else
```

```
        q[0] <= ~q[0];
```

```
end
```

```

always @(posedge q[0] or posedge rst)
begin
    if (rst)
        q[1] <= 0;
    else
        q[1] <= ~q[1];
end

```

```

always @(posedge q[1] or posedge rst)
begin
    if (rst)
        q[2] <= 0;
    else
        q[2] <= ~q[2];
end

```

```

always @(posedge q[2] or posedge rst)
begin
    if (rst)
        q[3] <= 0;
    else
        q[3] <= ~q[3];
end

```

endmodule

Test Bench:

```

Test b`timescale 1ns/1ps
module tb_async_up_counter_4bit;
reg clk, rst;
wire [3:0] q;
async_up_counter_4bit uut (
    .clk(clk),
    .rst(rst),
    .q(q)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_async_up_counter_4bit);
end
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;

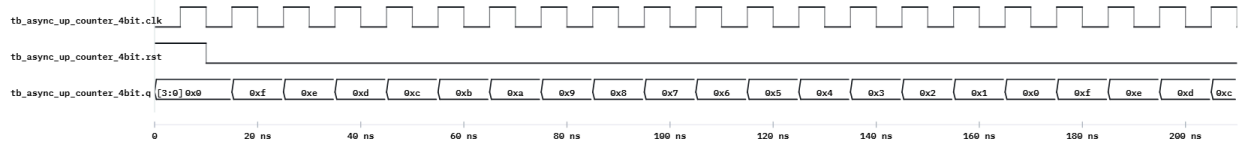
```

```

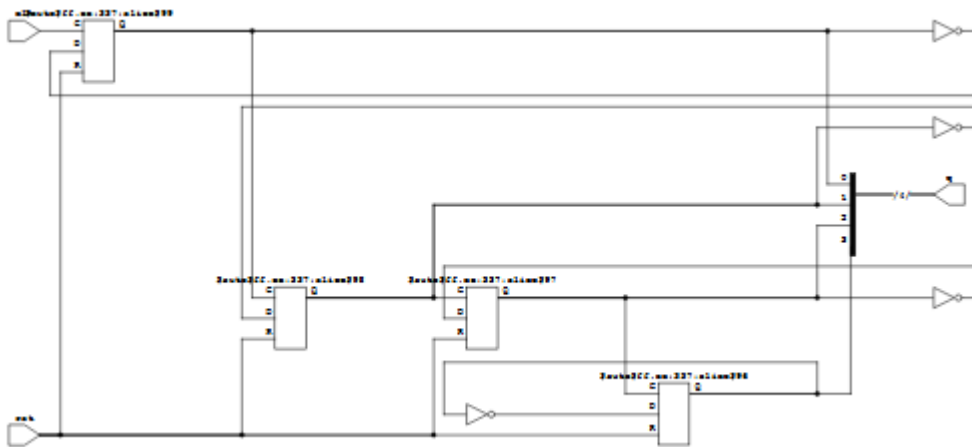
#10 rst = 0;
#200 $finish;
end
endmodule
ench :

```

Wave form :



Synthesis :



Task#7:4 bit asynchronous down counter

Verilog Code:

```

module async_down_counter_4bit(
    input clk,
    input rst,
    output reg [3:0] q
);
always @(posedge clk or posedge rst)
begin
    if (rst)
        q[0] <= 1'b1;
    else
        q[0] <= ~q[0];
end
always @(posedge q[0] or posedge rst)
begin
    if (rst)
        q[1] <= 1'b1;
    else

```

```

        q[1] <= ~q[1];
    end
    always @(posedge q[1] or posedge rst)
    begin
        if (rst)
            q[2] <= 1'b1;
        else
            q[2] <= ~q[2];
    end
    always @(posedge q[2] or posedge rst)
    begin
        if (rst)
            q[3] <= 1'b1;
        else
            q[3] <= ~q[3];
    end
end
endmodule

```

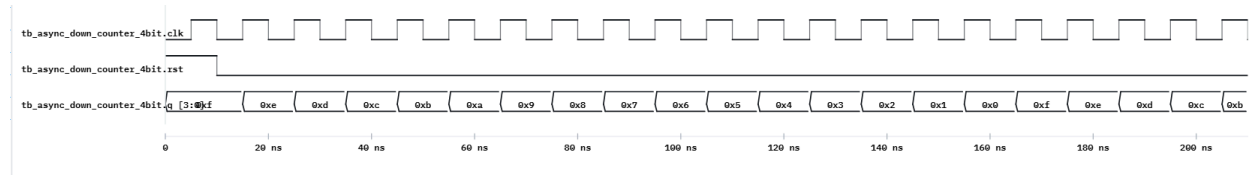
Test bench :

```

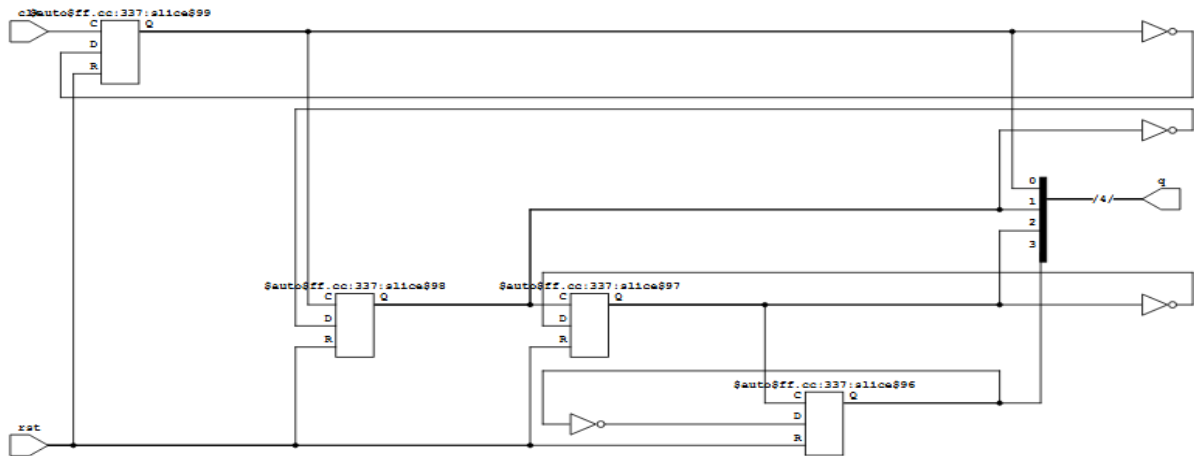
`timescale 1ns/1ps
module tb_async_down_counter_4bit;
    reg clk, rst;
    wire [3:0] q;
    async_down_counter_4bit uut(
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_async_down_counter_4bit);
    end
    always #5 clk = ~clk;
    initial begin
        clk = 0;
        rst = 1;
        #10 rst = 0;
        #200 $finish;
    end
end
endmodule

```


□ □



Synthesis



Task#8: BCD counter:

Verilog Code:

```
module bcd_counter(
    input clk,
    input rst,
    output reg [3:0] q
);
```

```
always @(posedge clk or posedge rst)
begin
```

```

if (rst)
    q <= 4'd0;
else if (q == 4'd9)
    q <= 4'd0;
else
    q <= q + 1'b1;
end

```

endmodule

Test bench :

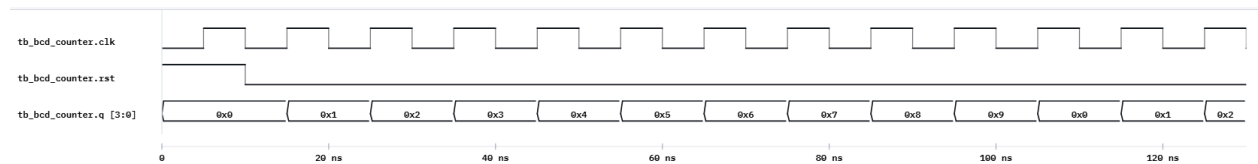
```
`timescale 1ns/1ps
module tb_bcd_counter;
reg clk, rst;
```

```

wire [3:0] q;
bcd_counter dut (
    .clk(clk),
    .rst(rst),
    .q(q)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_bcd_counter);
end
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    #10 rst = 0;
    #120 $finish;
end
endmodule

```

Wave form :



Tsk#9 MOD N counter:

Verilog Code:

```

module mod_n_counter #(
    parameter N = 6
)(
    input clk,
    input rst,
    output reg [2:0] q
);

always @(posedge clk or posedge rst)
begin
    if (rst)
        q <= 0;
    else if (q == N-1)
        q <= 0;
    else
        q <= q + 1;
end

```

```
end
```

```
endmodule
```

Test bench :

```
`timescale 1ns/1ps
module tb_mod_n_counter;
reg clk, rst;
wire [2:0] q;

mod_n_counter #(6) uut (
    .clk(clk),
    .rst(rst),
    .q(q)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_mod_n_counter);
end
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;

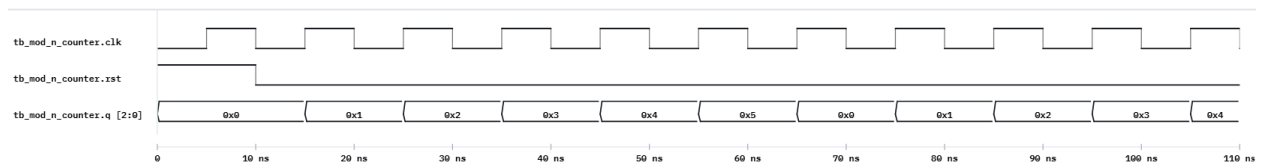
    #10 rst = 0;

    #100 $finish;
end
endmodule
```

Wave

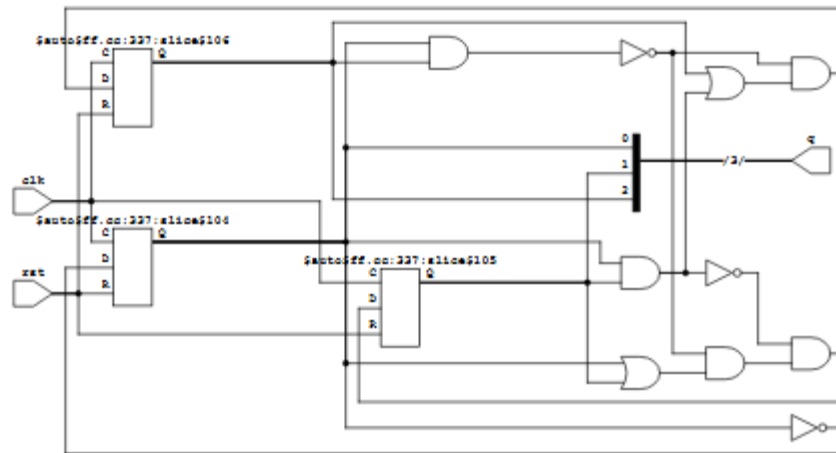
form

:



Synthesis

Gates · 5×AND 3×DFF_PP0 3×NOT 2×OR



Task#10:Frequency divider (100MHZ - 25 MHZ)

Verilog Code:

```
module freq_div_100_to_25(
    input clk,
    input rst,
    output reg clk_out
);
```

```
    reg [1:0] count;
```

```
    always @(posedge clk or posedge rst)
```

```
    begin
```

```
        if (rst) begin
```

```
            count <= 2'b00;
```

```
            clk_out <= 1'b0;
```

```
        end
```

```
        else begin
```

```
            count <= count + 1'b1;
```

```
            if(count == 2'b11)
```

```
                clk_out <= ~clk_out;
```

```
        end
```

```
    end
```

```
endmodule
```

Test bench :

```
`timescale 1ns/1ps
```

```

module tb_freq_div;

reg clk, rst;
wire clk_out;

freq_div_100_to_25 uut(
    .clk(clk),
    .rst(rst),
    .clk_out(clk_out)
);

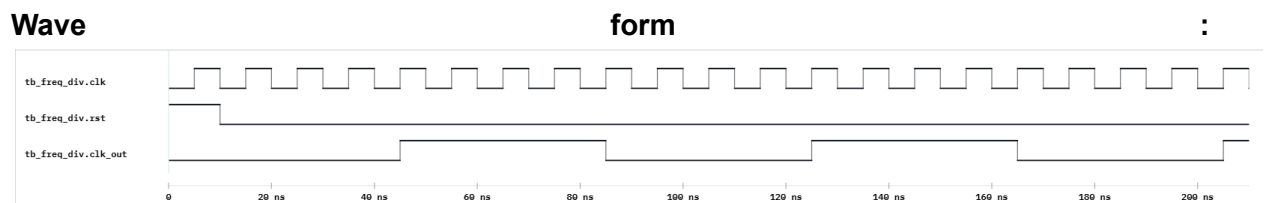
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_freq_div);
end

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    #10 rst = 0;

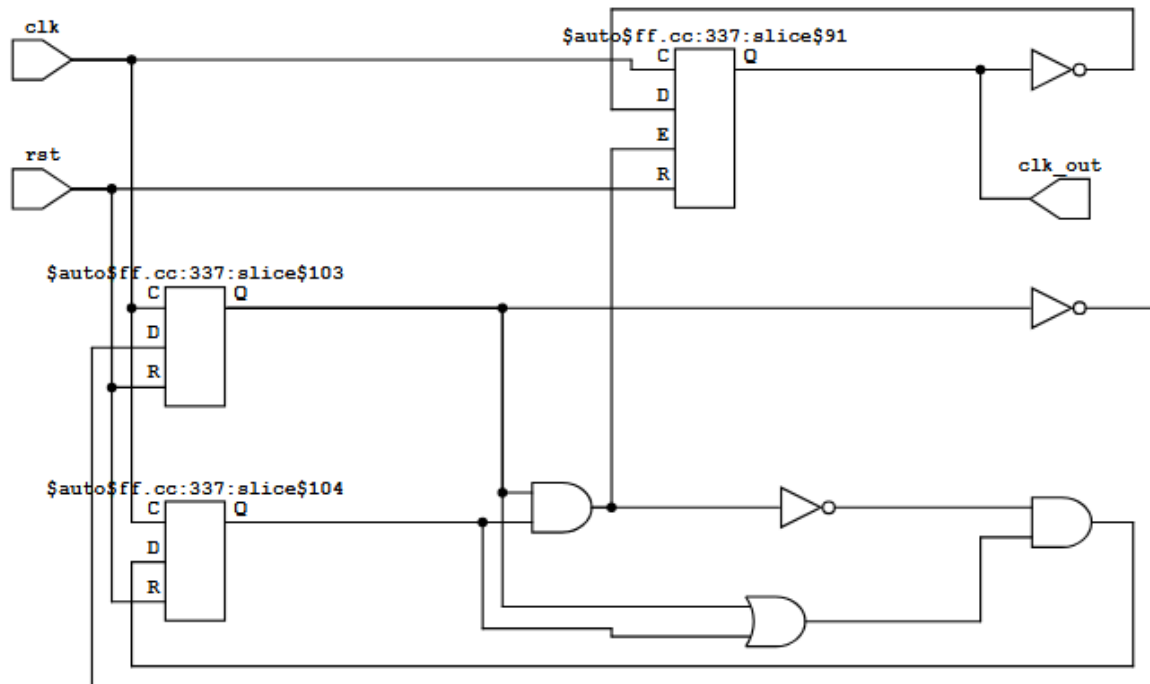
    #200 $finish;
end
endmodule

```



Synthesis

= fit Gates · 2×AND 1×DFFE_PP0P 2×DFF_PP0 3×NOT 1×OR



Task#11: Alu using pipeline

Verilog Code:

```
module pipelined_alu(
    input clk,
    input rst,
    input [31:0] A,
    input [31:0] B,
    input [2:0] ALU_Sel,
    output reg [31:0] Result
);
```

```
    reg [31:0] A_reg, B_reg;
    reg [2:0] ALU_Sel_reg;
    reg [31:0] alu_result;
```

```
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            A_reg <= 0;
            B_reg <= 0;
            ALU_Sel_reg <= 0;
        end
```

```

    else begin
        A_reg <= A;
        B_reg <= B;
        ALU_Sel_reg <= ALU_Sel;
    end
end
always @(posedge clk or posedge rst) begin
    if (rst)
        alu_result <= 0;
    else begin
        case(ALU_Sel_reg)
            3'b000: alu_result <= A_reg + B_reg;
            3'b001: alu_result <= A_reg - B_reg;
            3'b010: alu_result <= A_reg & B_reg;
            3'b011: alu_result <= A_reg | B_reg;
            3'b100: alu_result <= A_reg ^ B_reg;
            3'b101: alu_result <= ~(A_reg | B_reg);
            3'b110: alu_result <= (A_reg < B_reg) ? 32'd1 : 32'd0;
            default: alu_result <= 32'd0;
        endcase
    end
end

always @(posedge clk or posedge rst) begin
    if (rst)
        Result <= 0;
    else
        Result <= alu_result;
    end
endmodule

```

Test bench :

```

`timescale 1ns/1ps

module tb_pipelined_alu;

    reg clk;
    reg rst;
    reg [31:0] A;
    reg [31:0] B;
    reg [2:0] ALU_Sel;

    wire [31:0] Result;

```

```

pipelined_alu uut (
    .clk(clk),
    .rst(rst),
    .A(A),
    .B(B),
    .ALU_Sel(ALU_Sel),
    .Result(Result)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_pipelined_alu);
end

// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    A = 0;
    B = 0;
    ALU_Sel = 0;
    #10 rst = 0;

    // ADD
    A = 20;
    B = 10;
    ALU_Sel = 3'b000;
    #10;

    // SUB
    A = 30;
    B = 15;
    ALU_Sel = 3'b001;
    #10;

    // AND
    A = 12;
    B = 10;
    ALU_Sel = 3'b010;
    #10;

    // OR
    A = 12;
    B = 10;

```

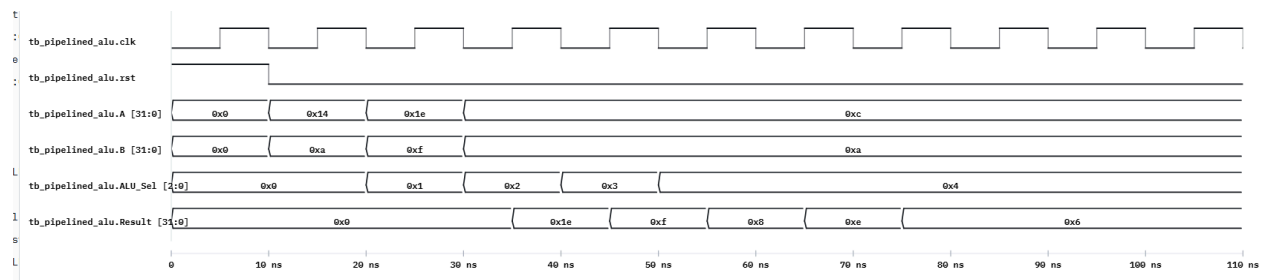


```

    ALU_Sel = 3'b011;
    #10;
    // XOR
    A = 12;
    B = 10;
    ALU_Sel = 3'b100;
    #10;
    #50;
    $finish;
end
initial begin
    $monitor("Time=%0t A=%d B=%d ALU_Sel=%b Result=%d",
        $time, A, B, ALU_Sel, Result);
end
endmodule

```

Wave form :



Task#12 : Multiplier using pipeline

Verilog Code:

```

module pipelined_multiplier(
    input clk,
    input rst,
    input [3:0] A,
    input [3:0] B,
    output reg [7:0] P
);

```

```

    reg [3:0] A_reg, B_reg;
    reg [7:0] mult_reg;

```

```

always @(posedge clk or posedge rst)
begin
    if(rst) begin
        A_reg <= 0;
        B_reg <= 0;

```

```

    end
    else begin
        A_reg <= A;
        B_reg <= B;
    end
end

always @(posedge clk or posedge rst)
begin
    if(rst) begin
        mult_reg <= 0;
        P <= 0;
    end
    else begin
        mult_reg <= A_reg * B_reg;
        P <= mult_reg;
    end
end
endmodule

```

Test bench :

```

`timescale 1ns/1ps

module tb_pipelined_multiplier;

    reg clk, rst;
    reg [3:0] A, B;
    wire [7:0] P;

    pipelined_multiplier uut(
        .clk(clk),
        .rst(rst),
        .A(A),
        .B(B),
        .P(P)
    );

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_pipelined_multiplier);
    end

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
    end
endmodule

```

```
A = 0;
B = 0;
```

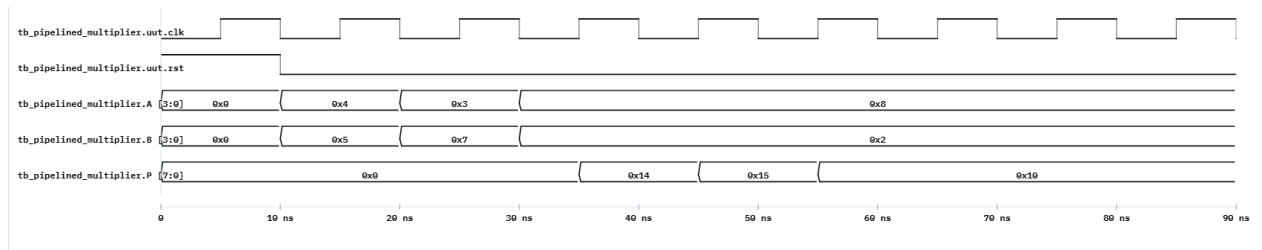
```
#10 rst = 0;
A = 4; B = 5; #10;
A = 3; B = 7; #10;
A = 8; B = 2; #10;
```

```
#50 $finish;
end
endmodule
```

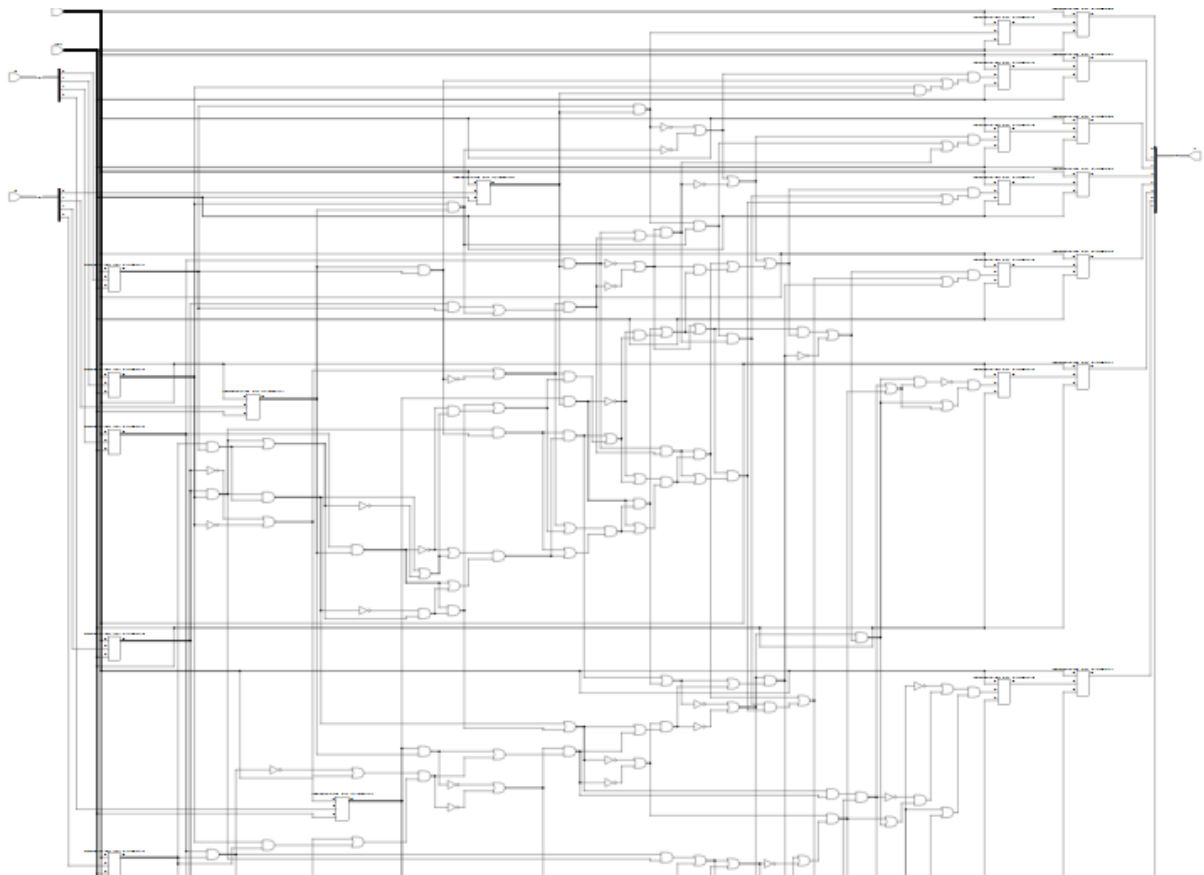
Wave

form

:



Synthesis



Task#13: Johnson counter

Verilog Code:

```
module johnson_counter(  
    input clk,  
    input rst,  
    output reg [3:0] q  
);  
always @(posedge clk or posedge rst)  
begin  
    if (rst)  
        q <= 4'b0000;  
    else  
        q <= {q[2:0], ~q[3]};  
    end  
end  
endmodule
```

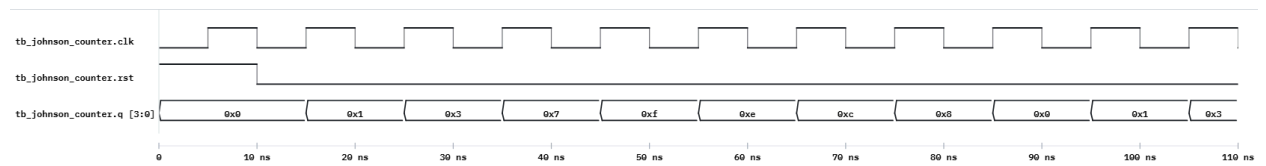
Test bench :

```
`timescale 1ns/1ps  
module tb_johnson_counter;  
    reg clk, rst;  
    wire [3:0] q;  
    johnson_counter uut(  
        .clk(clk),  
        .rst(rst),  
        .q(q)  
    );  
    initial begin  
        $dumpfile("dump.vcd");  
        $dumpvars(0, tb_johnson_counter);  
    end  
    always #5 clk = ~clk;  
    initial begin  
        clk = 0;  
        rst = 1;  
        #10 rst = 0;  
        #100 $finish;  
    end  
    end  
    initial begin  
        $monitor("Time=%0t Q=%b", $time, q);  
    end  
end  
endmodule
```

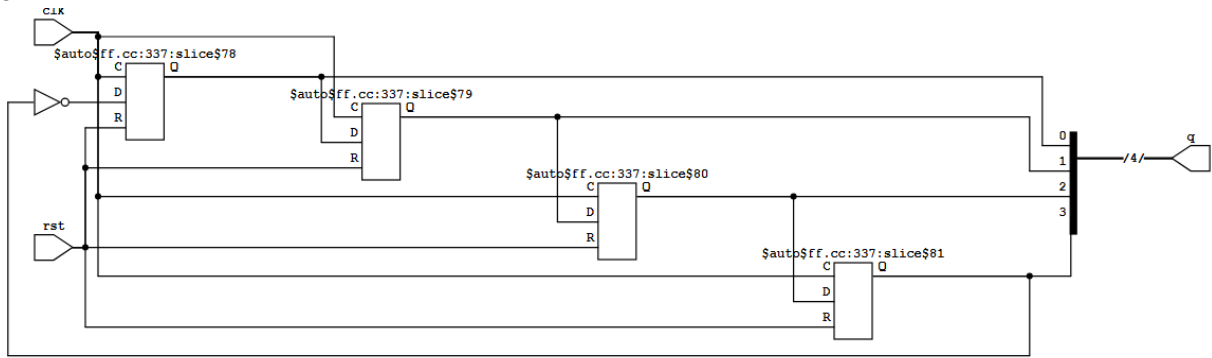
Wave

form

:



Synthesis



Task#14 Ring counter

Verilog Code:

```
module ring_counter(  
    input clk,  
    input rst,  
    output reg [3:0] q  
);  
always @(posedge clk or posedge rst)  
begin  
    if (rst)  
        q <= 4'b0001;  
    else  
        q <= {q[2:0], q[3]};  
    end  
end  
endmodule
```

Test Bench:

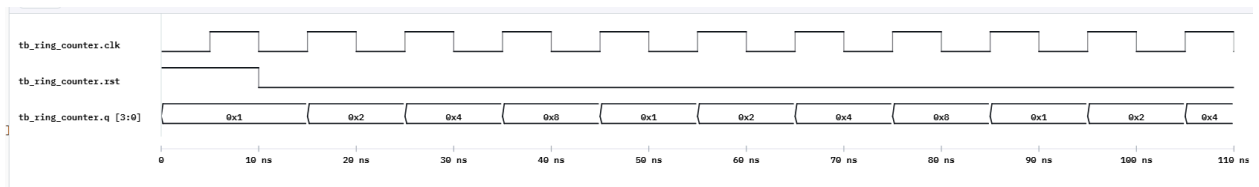
```
`timescale 1ns/1ps  
module tb_ring_counter;  
    reg clk, rst;  
    wire [3:0] q;  
    ring_counter uut(  
        .clk(clk),  
        .rst(rst),  
        .q(q)  
    );  
    initial begin  
        $dumpfile("dump.vcd");
```

```

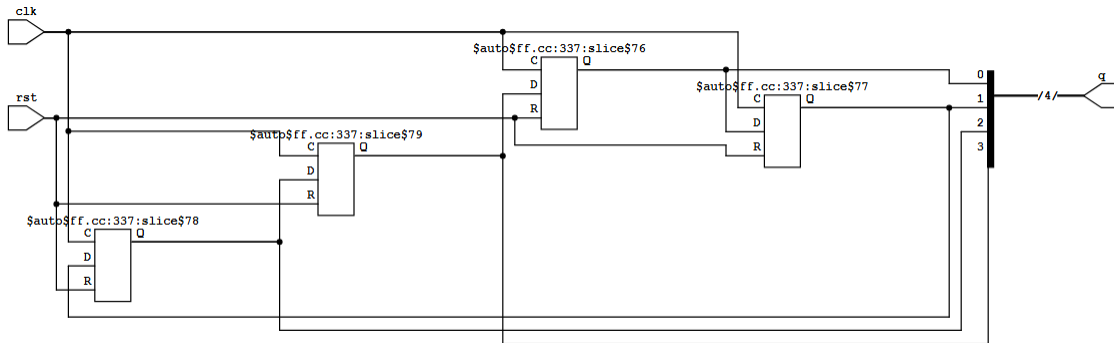
    $dumpvars(0, tb_ring_counter);
end
always #5 clk = ~clk;
initial begin
    clk = 0;
    rst = 1;
    #10 rst = 0;
    #100 $finish;
end
initial begin
    $monitor("Time=%0t Q=%b", $time, q);
end
endmodule

```

WaveForm:



Synthesis



Task#13 SISO

Verilog Code:

```

module siso(
    input clk,
    input rst,
    input serial_in,
    output reg serial_out
);

```

```

    reg [3:0] shift_reg;

```

```

    always @(posedge clk or posedge rst)

```

```

begin
    if (rst)
        shift_reg <= 4'b0000;
    else
        shift_reg <= {shift_reg[2:0], serial_in};
    end

always @(*)
begin
    serial_out = shift_reg[3];
end

endmodule

```

Test bench :

```

`timescale 1ns/1ps

module tb_asiso;

reg clk, rst, serial_in;
wire serial_out;

siso uut(
    .clk(clk),
    .rst(rst),
    .serial_in(serial_in),
    .serial_out(serial_out)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_asiso);
end

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    serial_in = 0;

    #10 rst = 0;

    serial_in = 1; #10;

```

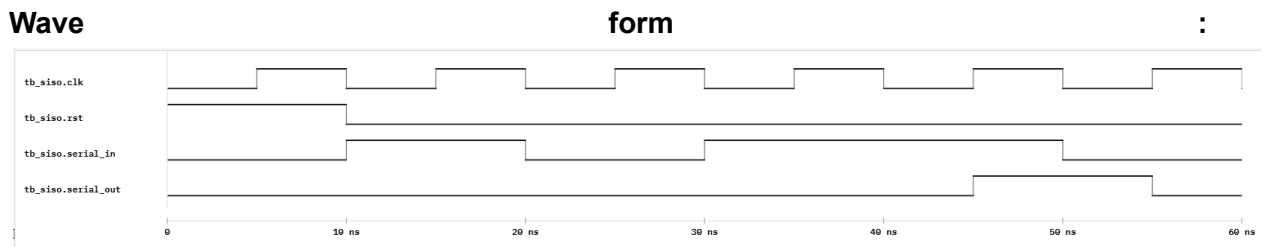
```

    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;
    serial_in = 0; #10;

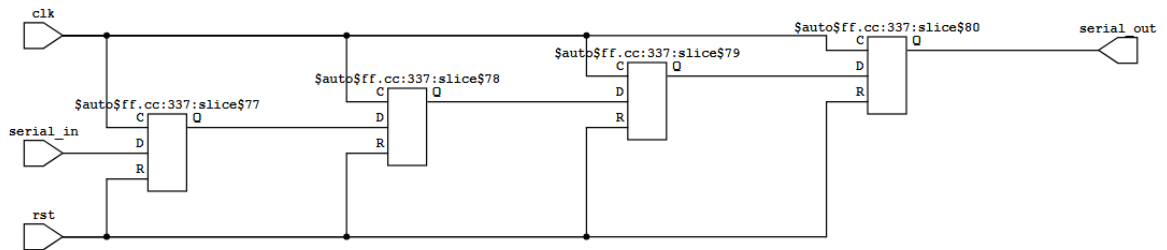
    $finish;
end

endmodule

```



Synthesis



Task#14 SIPO

Verilog Code:

```

module sipo(
    input clk,
    input rst,
    input serial_in,
    output reg [3:0] q
);

always @(posedge clk or posedge rst)
begin
    if (rst)
        q <= 4'b0000;
    else
        q <= {q[2:0], serial_in};
end

endmodule

```


Test bench :

```
`timescale 1ns/1ps

module tb_sipo;

reg clk, rst, serial_in;
wire [3:0] q;

sipo uut(
    .clk(clk),
    .rst(rst),
    .serial_in(serial_in),
    .q(q)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_sipo);
end

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    serial_in = 0;

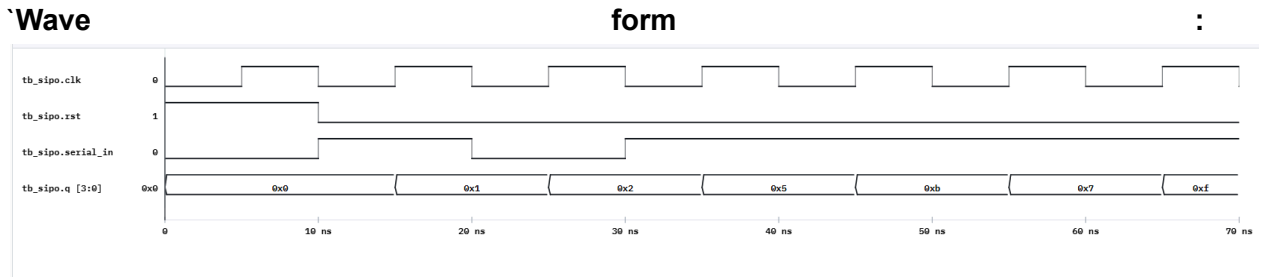
    #10 rst = 0;

    serial_in = 1; #10;
    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;

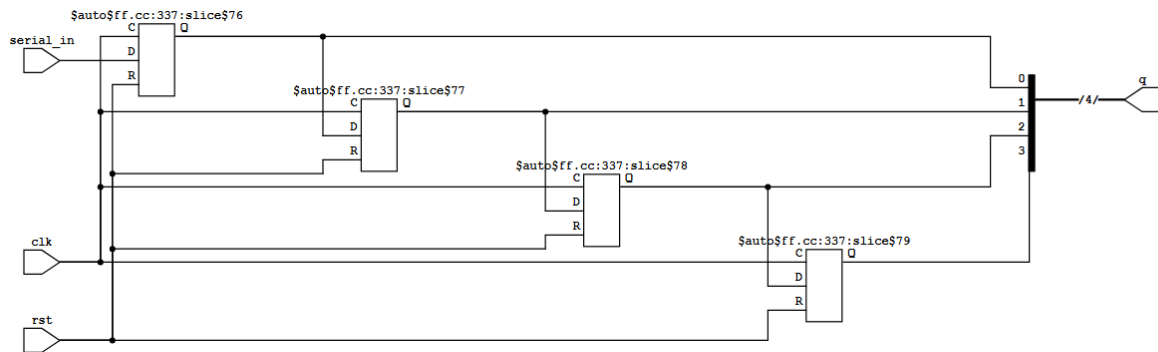
    #20 $finish;
end

initial begin
    $monitor("Time=%0t Serial_In=%b Q=%b",
        $time, serial_in, q);
end

endmodule
```



Synthesis:



Task#15 PISO

Verilog Code:

```

module piso(
    input clk,
    input rst,
    input load,
    input [3:0] parallel_in,
    output reg serial_out
);

reg [3:0] shift_reg;

always @(posedge clk or posedge rst)
begin
    if (rst)
        shift_reg <= 4'b0000;

    else if (load)
        shift_reg <= parallel_in;

    else
        shift_reg <= {shift_reg[2:0], 1'b0};
end

always @(*)

```

```
begin
    serial_out = shift_reg[3];
end

endmodule
```

Test bench :

```
`timescale 1ns/1ps

module tb_piso;

reg clk, rst, load;
reg [3:0] parallel_in;
wire serial_out;

piso dut(
    .clk(clk),
    .rst(rst),
    .load(load),
    .parallel_in(parallel_in),
    .serial_out(serial_out)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb_piso);
end

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    load = 0;
    parallel_in = 4'b0000;

    #10 rst = 0;

    load = 1;
    parallel_in = 4'b1011; // Load data
    #10;

    load = 0;           // Start shifting
```

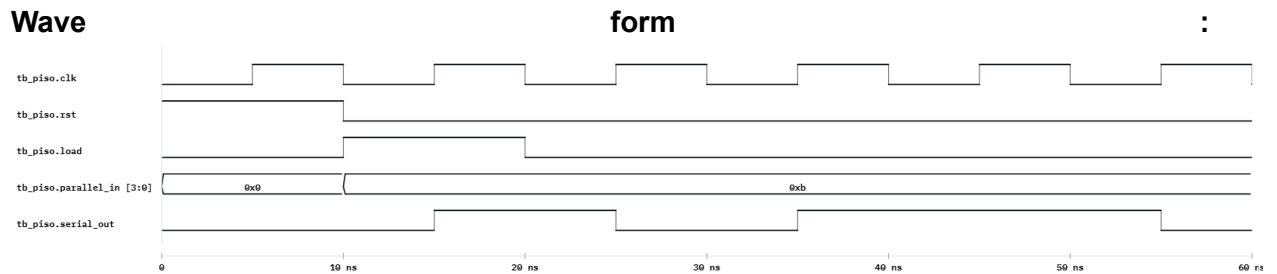
```

#40;

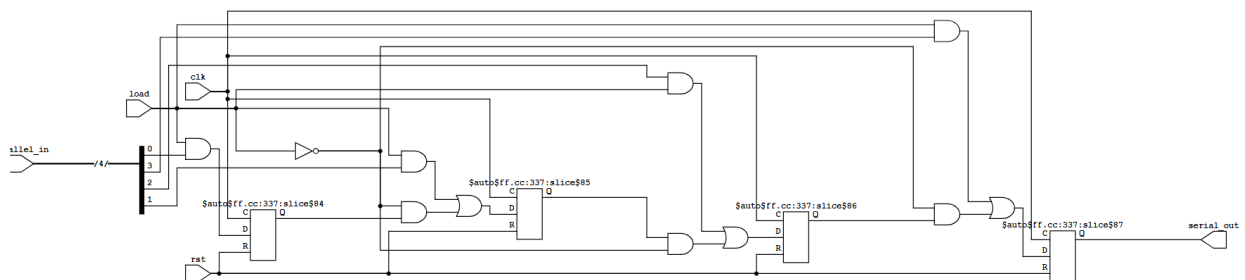
$finish;
end

endmodule

```



Synthesis



Task#16 PIPO

Verilog Code:

```

module pipo(
    input clk,
    input rst,
    input [3:0] parallel_in,
    output reg [3:0] parallel_out
);

always @(posedge clk or posedge rst)
begin
    if (rst)
        parallel_out <= 4'b0000;
    else
        parallel_out <= parallel_in;
end

endmodule

```

Test bench :

```
`timescale 1ns/1ps
```

```
module tb_pipo;
```

```
reg clk;
```

```
reg rst;
```

```
reg [3:0] parallel_in;
```

```
wire [3:0] parallel_out;
```

```
pipo uut(
```

```
    .clk(clk),
```

```
    .rst(rst),
```

```
    .parallel_in(parallel_in),
```

```
    .parallel_out(parallel_out)
```

```
);
```

```
initial begin
```

```
    $dumpfile("dump.vcd");
```

```
    $dumpvars(0, tb_pipo);
```

```
end
```

```
always #5 clk = ~clk;
```

```
initial begin
```

```
    clk = 0;
```

```
    rst = 1;
```

```
    parallel_in = 4'b0000;
```

```
    #10 rst = 0;
```

```
    parallel_in = 4'b1010; #10;
```

```
    parallel_in = 4'b1100; #10;
```

```
    parallel_in = 4'b1111; #10;
```

```
    $finish;
```

```
end
```

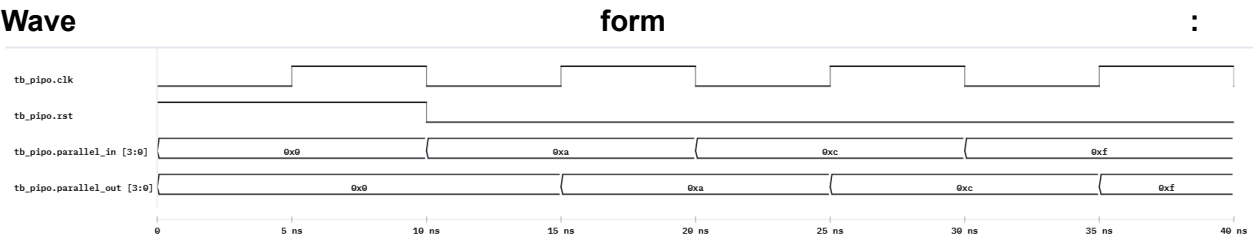
```
initial begin
```

```
    $monitor("Time=%0t Input=%b Output=%b",
```

```
        $time, parallel_in, parallel_out);
```

```
end
```

```
endmodule
```



Synthesis

